12 June 2026



Università degli Studi di Padova

Università Degli Studi di Padova

Cyber-Physical Systems and IoT Security 2025/2026

Alessandro Tigani Sava, student ID: 2198854

First project report

Error Handling of In-vehicle Networks Makes Them Vulnerables

1 Objectives

This project aims to simulate and analyze the bus-off attack on in-vehicle networks, specifically the Controller Area Network (CAN) bus.
The focus is on modeling how adversarial ECUs exploit the error-handling mechanisms to disconnect legitimate ECUs by forcing them into a Bus-off state.
The simulation, implemented in Python, recreates the attack dynamics described in the reference paper, including the behavior of the Transmit Error Counter (TEC) and the asymmetry in error state manipulation between adversarial and normal ECUs.
The results are validated against the original study to ensure the model accurately reflects real-world attack behavior.

2 Setup

The original paper implemented the attack using real hardware components, but this project recreates the scenario entirely in Python by simulating both the ECUs and the CAN bus.
To run the simulation, a virtual environment is recommended.
The required libraries, listed in the `requirements.txt` file, can be installed using standard Python commands.
The main library used in the project is `matplotlib`, wich is necessary to plot the results and compare them with the ones presented in the paper.
Once the dependencies are in place, executing the `main.py` script initiates the simulation and generates the results for analysis.

```
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
py main.py
```

3 Experiments

The main script contains three classes used to simulate the ECUs, the CAN Bus and a Packet that can be sent by the ECU.
The ECU class contains a function used to simulate the behaviour of the node.

Packet	
Parameter	**Type**
ecu_id	int
id	int
payload	list
Method	**Return type**
__init__(ecu_id, id, payload)	
to_Array()	list

Table 1: Packet class

The Packet class in Table 1 simulates the data transmitted by an ECU over the CAN Bus. It is a minimal implementation tailored for this simulation, and it omits the full complexity of real messages.

Each instance contains three key attributes: the sender's ECU ID, a message ID, and a data payload.
Additionally, the class includes a method to format messages as arrays for logging.

ECU	
Parameter	**Type**
adversary_target_id	int
ecu_id	int
tec	int
error_flag	bool
retransmission	bool
status	str
tec_history	list
sent_packets	dict
Method	**Return type**
__init__(ecu_id, adversary_target_id)	
update_status()	
send_packet(packet_id, payload, canbus, retransmission)	
check_duplicate_ids(packet_id, canbus)	
update_tec(packet_id, canbus)	
run_ecu(packet_id, payload, canbus, barrier)	

Table 2: ECU class

The ECU class, depicted in Table 2, simulates an ECU node, which can function as either a standard participant or an adversary that disrupts communication by manipulating its error state.
Each node maintains a Transmit Error Counter (TEC), which updates dynamically based on the success or failure of message transmissions.
Additionally, the node monitors the bus for duplicate message IDs and adjusts its error state accordingly.
Initially, all nodes start in the "Error Active" state and transition to "Error Passive" or "Bus-off" as their TEC changes.
Each node continuously simulate message transmission, duplicate detection, and TEC updates until the simulation ends or they enter the Bus-off state.
The core of the simulation is the `run_process` function, which implements the algorithm for each node.
It operates in a loop while the simulation remains active, the node stays connected, and at least two ECUs are present on the bus.
If a node enters the Bus-off state, it disconnects from the bus.
The function uses a threading barrier to synchronize message transmission, sending either new messages or retransmissions if duplicates are detected.
It then checks for duplicate message IDs and updates the TEC based on whether the node is an adversary or a normal ECU, adjusting the error state accordingly.
This process try to replicates the behavior of real ECUs on a CAN bus, including error handling and message retransmission.

CANBus	
Parameter	**Type**
ecus	list
packet_log	list
lock	Lock
Method	**Return type**
__init__()	
connect_ecu(ecu)	
disconnect_ecu(ecu)	
is_ecu_connected(ecu)	bool

Table 3: CANBus class

The CANBus class, illustrated in Table 3, manages a list of connected ECUs and a log of all messages transmitted on the bus.
It provides methods for connecting and disconnecting ECUs and checking their connection status
To ensure thread-safe operations, the class employs a lock, allowing multiple nodes to interact with the bus concurrently without risking data corruption.
Like the Message class, this is a streamlined implementation that includes only the essential features required for the simulation.

4 Results

The simulation replicated the bus-off attack dynamics described in the reference paper, demonstrating how an adversarial ECU can exploit the CAN bus error-handling mechanisms to force a legitimate ECU into the Bus-off state.
As shown in Figure 1, the Transmit Error Counter (TEC) for both ECUs initially increases by 8 during conflicting message retransmissions.

Once the victim ECU TEC reaches the Error Passive threshold (128), the attacker ECU begins manipulating its error state by sending messages with non-dominant bits.
This causes the victim's TEC to rise rapidly while the attacker's TEC decreases.
Specifically, the victim's TEC increases by 7 since a failed transmission (+8) is followed by a successful retransmission (-1), while the attacker's TEC decreases by 1 because of the successful transmission.
This asymmetry ultimately forces the victim into the Bus-off state, disconnecting it from the bus, while the attacker remains in the Error Active state.

Comparing the simulation results with the paper's experimental results in Figure 2 confirms that the implemented model accurately captures the attack's behavior.
Even if the TEC value increase rate differ slightly due to simulation-specific parameters such as message delay times, the overall trends closely align with the theoretical and practical findings of the original study.
This comparison shows that the Python-based simulation, despite its simplified design, effectively replicates the real-world behavior of CAN bus error handling.

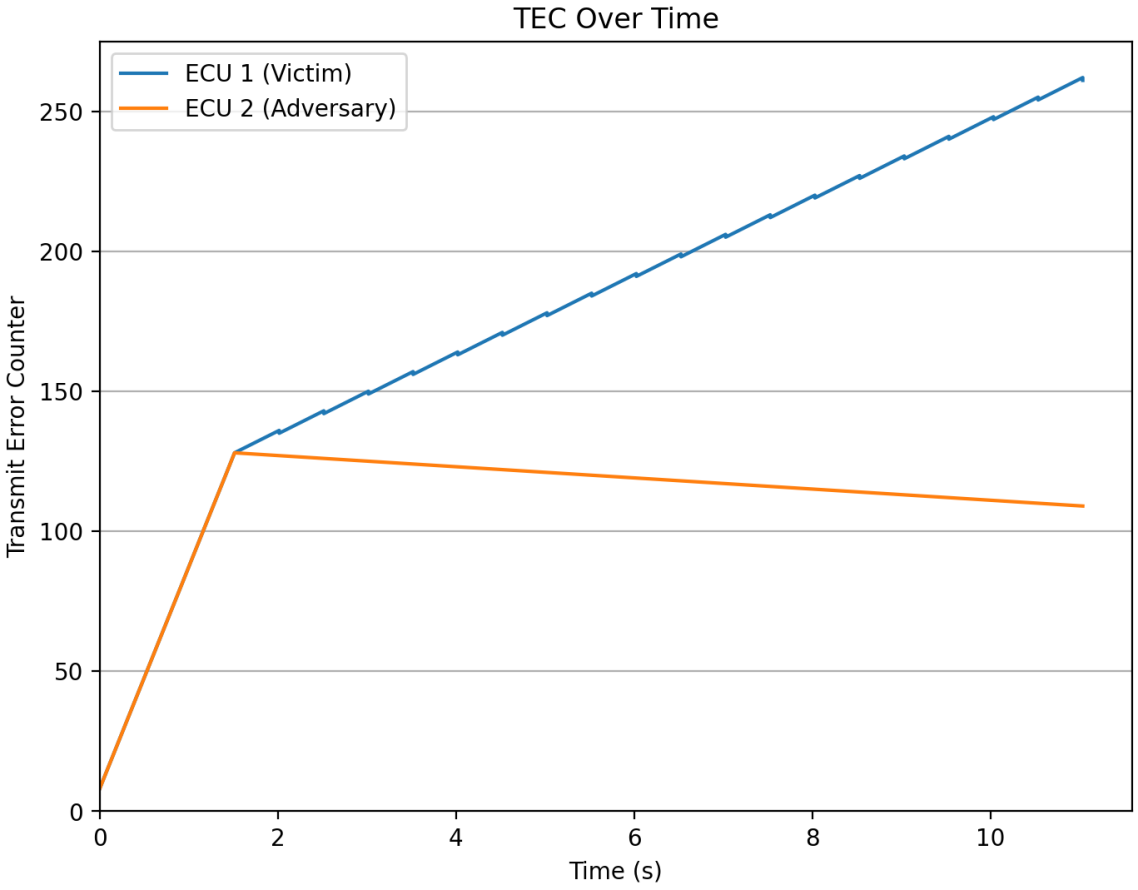


Figure 1: Simulation results

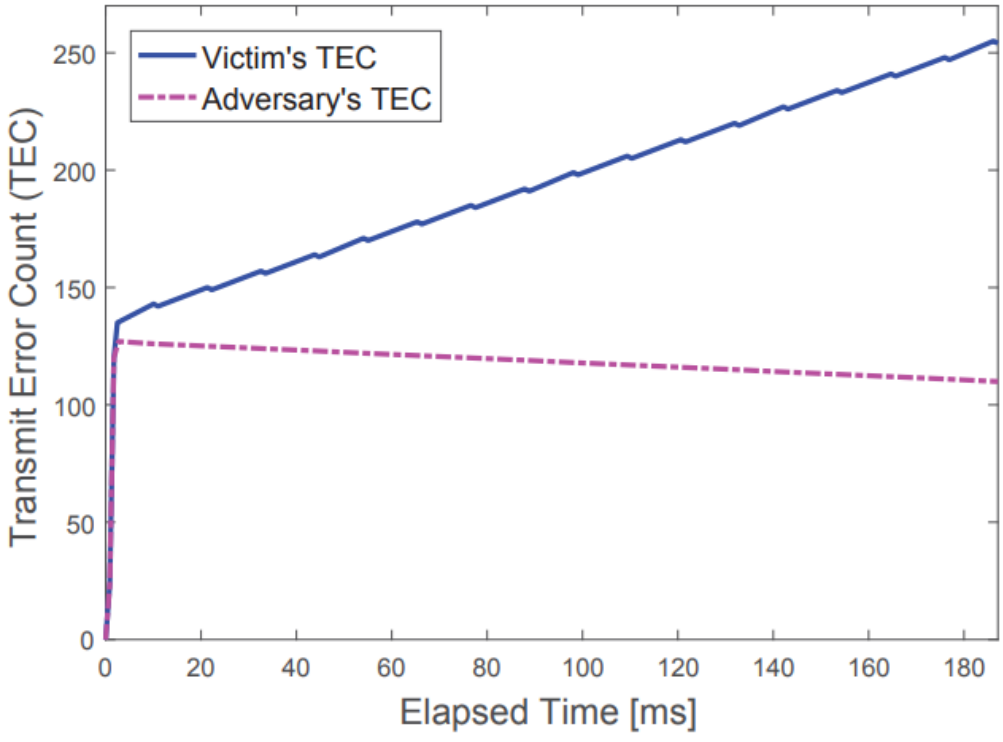


Figure 2: Paper results